

The program needs to track only the latest two numbers of the sequence at a time. Since the first two numbers in the Fibonacci sequence are defined as 1, we store 1 in variables `a` and `b` ❶. Inside the `for` loop, the next number in the sequence is calculated by adding `a` and `b` ❷, which becomes the next value of `b`, while `a` obtains the previous value of `b`. By the time the loop is finished, `b` contains the n th Fibonacci number, so it is returned.

The Recursive Fibonacci Algorithm

Calculating Fibonacci numbers involves a recursive property. For example, if you want to calculate the 10th Fibonacci number, you add the ninth and eighth Fibonacci numbers together. To calculate those Fibonacci numbers, you add the eighth and seventh, then the seventh and sixth Fibonacci numbers. A lot of repeat calculations occur: notice that adding the ninth and eighth Fibonacci numbers involves calculating the eighth Fibonacci number again. You continue this recursion until you reach the base case of the first or second Fibonacci number, which is always 1.

The recursive Fibonacci function is in this *fibonacciByRecursion.py* Python program:

```
def fibonacci(nthNumber):
    print('fibonacci(%s) called.' % (nthNumber))
    if nthNumber == 1 or nthNumber == 2: ❶
        # BASE CASE
        print('Call to fibonacci(%s) returning 1.' % (nthNumber))
        return 1
    else:
        # RECURSIVE CASE
        print('Calling fibonacci(%s) and fibonacci(%s).' % (nthNumber - 1, nthNumber - 2))
        result = fibonacci(nthNumber - 1) + fibonacci(nthNumber - 2)
        print('Call to fibonacci(%s) returning %s.' % (nthNumber, result))
        return result

print(fibonacci(10))
```

This *fibonacciByRecursion.html* file has the equivalent JavaScript program: